# Bridge Product Requirements Document

**Student talent marketplace | Version 1.0 | 2 November 2026**

## Executive summary

Bridge is a two-sided student talent marketplace concept. The Stitch screens support a core product in which students create skill profiles, browse paid tasks, submit proposals and portfolio highlights, and communicate with clients after a match. Task owners publish work, review applicants, and choose candidates. The product should first make that core hiring loop understandable and reliable. Payment processing, dispute resolution, ratings policy, and live messaging infrastructure are not specified by the screens and remain out of this release definition until confirmed.

## Product context

The source is the Google Stitch project "Bridge: Student Talent Marketplace." It contains mobile screens for role selection, login, student onboarding, task discovery and details, applications, profiles, client task posting, applicant review, matches, and chat. This PRD records the visible product direction and proposes requirements needed to implement it. Proposed decisions are marked as such; no revenue model or measurable business target was provided.

## Users and needs

| User | Need evidenced by the screens |
|---|---|
| Student / applicant | Present relevant skills and portfolio, find suitable tasks, understand terms, apply, and track application or work status. |
| Task owner / freelancer-side user | Describe work, set required skills and budget/deadline, compare candidates, accept or reject applicants, and communicate with a selected candidate. |
| Bridge operator | Maintain safe accounts, consistent task and application states, and trustworthy marketplace information. Operator tooling is a proposed operational need, not shown in the screens. |

## Goals and non-goals

### Goals

- Let a student create and maintain a profile with skills, experience level, availability, and portfolio link.
- Let a student browse available tasks, review task details, and submit a proposal with selected portfolio highlights.
- Let a task owner create a task and review its applicants.
- Make application status and accepted matches visible to the involved users.
- Support private conversation between eligible matched participants.
- Enforce ownership and membership permissions for private marketplace data.

### Non-goals for the initial scope

- Processing payments, escrow, invoicing, payouts, refunds, or tax reporting. The UI displays budget and pending-payment figures, but no payment flow or provider is shown.
- Defining a rating, verification, recommendation, or matching algorithm.
- Attachments, video calls, typing indicators, push notifications, or other realtime features beyond visible text chat.
- Employer verification, background checks, moderation console, or dispute workflow until policy is defined.
- Multi-platform scope beyond the mobile product represented by the current board.

# Product requirements

## Identity and profile

| ID | Requirement | Acceptance condition |
|---|---|---|
| PR-01 | A visitor can choose Student or Freelancer mode. | The selected mode controls relevant onboarding/navigation. Whether one account can use both modes is an open decision. |
| PR-02 | A user can sign in with email/password or Google. | Invalid credentials fail safely; Google identity is verified by the server. Signup and recovery requirements need confirmation. |
| PR-03 | A student can create and edit name, skills, experience level, weekly availability, and portfolio URL. | Saved values reload consistently; invalid values return field-level errors. |
| PR-04 | A user can view a public profile with approved profile fields. | Private account data is not included in public profile responses. |

## Task discovery and application

| ID | Requirement | Acceptance condition |
|---|---|---|
| PR-05 | Students can browse available tasks and open task details. | Only tasks in the agreed public state appear; task detail includes the visible description, skills, budget, deadline, and owner summary. |
| PR-06 | A student can submit a proposal and choose portfolio highlights. | The server validates task eligibility and prevents unintended duplicate applications. |
| PR-07 | A student can see their application states. | Only the student's own applications are returned; state labels match the agreed lifecycle. |

## Task creation and review

| ID | Requirement | Acceptance condition |
|---|---|---|
| PR-08 | A task owner can create a task with title, description, required skills, budget, and deadline. | Required fields, currency, budget rules, and deadline policy are validated server-side. |
| PR-09 | A task owner can review applicant summaries and accept or reject candidates. | Only the task owner can view the applicant list or decide; the state change is durable and idempotent. |
| PR-10 | Task state and applicant state remain consistent. | Concurrent decisions do not produce a state combination forbidden by the agreed hiring policy. |

## Matches and conversation

| ID | Requirement | Acceptance condition |
|---|---|---|
| PR-11 | Users can view the relevant match/application groups shown in the design. | Each user sees only their own relationship state; exact group meanings are confirmed before release. |
| PR-12 | Eligible participants can read and send text messages in a conversation. | Non-members cannot read or send; messages persist in stable order. Eligibility rule and realtime needs are open decisions. |

# Primary user journeys

1. **Student setup:** choose Student to authenticate to complete profile and skills to view student home.
2. **Apply:** browse tasks to view task details to write proposal to select portfolio highlights to submit to track decision.
3. **Hire:** choose Freelancer/task-owner mode to create task to review applicants to accept/reject to open conversation with eligible participant.
4. **Communicate:** open an eligible conversation to read history to send text message to view task/project context if its behavior is defined.

# UX and content requirements

- Show explicit loading, empty, success, and failure states for remote lists and writes.
- Explain why an application cannot be submitted when a task is closed or the user is ineligible.
- Display budget with currency and deadline with an unambiguous timezone or local-time convention.
- Use clear application states and accessible labels for accept/reject, submit, and message controls.

- Avoid presenting "verified," ratings, earnings, or completion percentages until their definitions and sources are approved.

## Success measures

No numeric targets were supplied. Instrument these candidate measures after definitions and privacy review: profile completion rate; task-detail-to-application conversion; time from task publication to first qualified application; applicant decision time; acceptance rate; conversation activation after acceptance; and task completion rate if completion status is added. Establish baseline and target values with the business owner before launch.

## Dependencies and assumptions

- The Stitch board is a concept prototype, not a complete production specification.
- A task owner is authorized to see applications to their task; applicants can see only their own submissions.
- The API and schema are implementation proposals in the separate TRD/backend handoff.
- The marketplace's country, currency, age eligibility, student verification, operating model, moderation policy, and payment scope have not been established.

## Open product decisions

1. Are Student and Freelancer exclusive account types or switchable modes on one account?
2. What does the draft/review control on task posting mean, and who can publish a task?
3. Can one task hire multiple applicants? What happens to remaining applications after acceptance?
4. Can proposals be edited or withdrawn?
5. How are "matches" computed and what do each visible status group mean?
6. What do pending payment and earnings mean, and are payments in scope?
7. What are the definitions for ratings, verification, completion rate, active task, and project goal/progress?
8. What signup, verification, password recovery, reporting, blocking, deletion, and retention flows are required?

## Release readiness

Before implementation sign-off, the product owner should approve account modes, task/application lifecycle, hiring multiplicity, match-state definitions, public profile fields, payment scope, and chat eligibility. The team can then convert PR-01 through PR-12 into client/server stories and acceptance tests.

**Source:** Bridge: Student Talent Marketplace, Google Stitch project 14483777616722284649. Requirements beyond visible screens are proposals or open decisions.